

《Object-oriented Technology》

Course Project Report

Title:	**STUDY NOTES MANAGER**
Major:	**SOFTWARE ENGINEERING**
Name:	**MD TASNIMUL HASAN**
Student ID:	**202480090104**

1. Project Overview

Study Notes Manager with AI Assistance is a full-stack web application that empowers students to create, organize, and intelligently enhance their study notes using cutting-edge AI capabilities from the DeepSeek API. The platform transforms passive note-taking into an active, AI-driven learning experience.

1.1 Project Goals

- Provide a clean, modern interface for creating and managing study notes
- Integrate AI to automate title generation, summarization, and concept extraction
- Enable quiz generation from notes with automatic scoring and feedback
- Support file uploads (TXT, PDF) for seamless note creation
- Implement persistent SQLite storage with full CRUD operations
- Deliver a responsive design that works on desktop and mobile

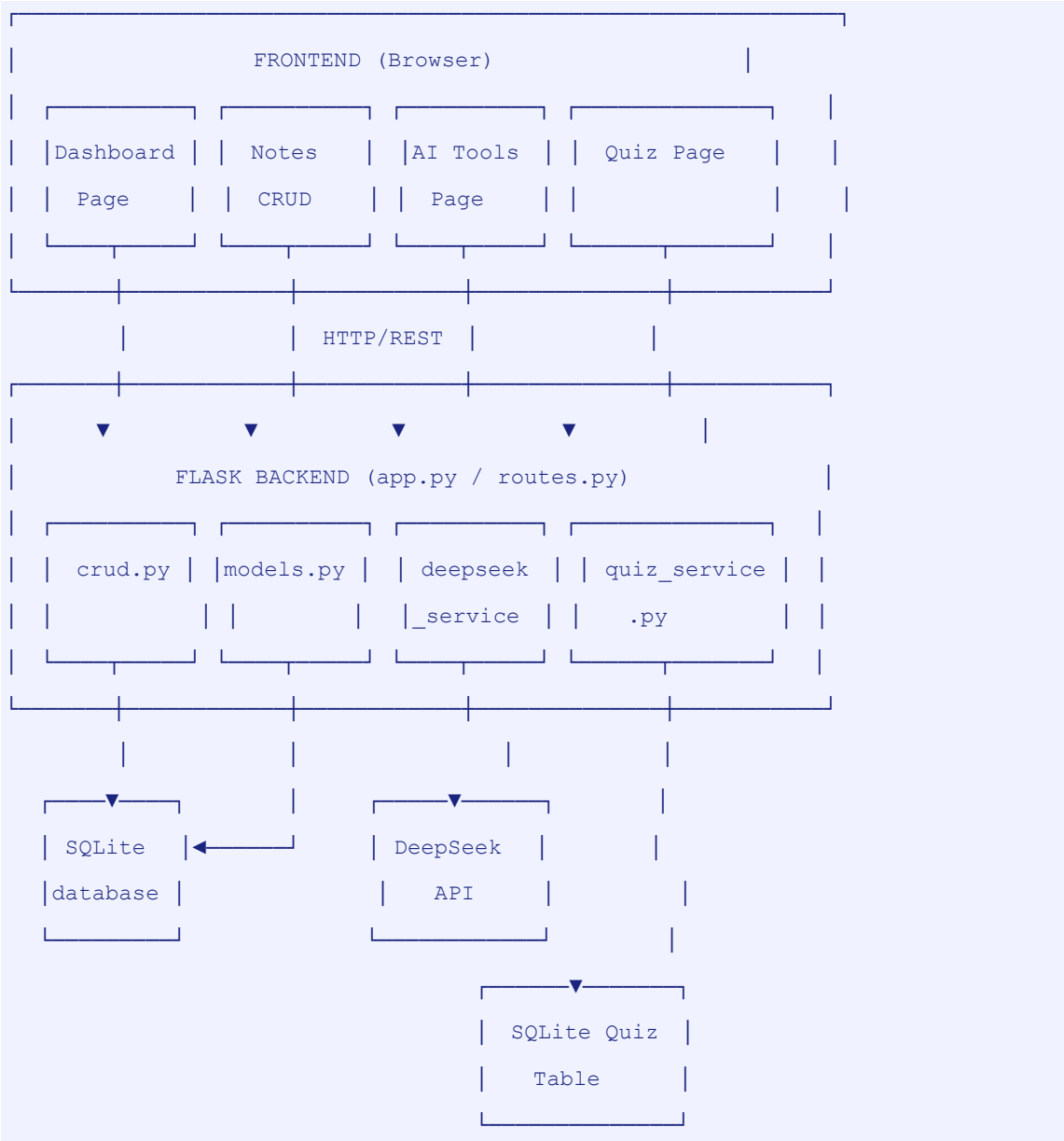
1.2 Technology Stack

Layer	Technology	Purpose
Frontend	HTML5 + CSS3 + JavaScript	User interface and dynamic interactions
Backend	Python Flask	REST API, routing, business logic
Database	SQLite	Persistent storage for notes and quizzes
AI Service	Deepseek API	Title gen, summaries, key concepts, quizzes
PDF Parsing	PyPDF2	Text extraction from uploaded PDFs

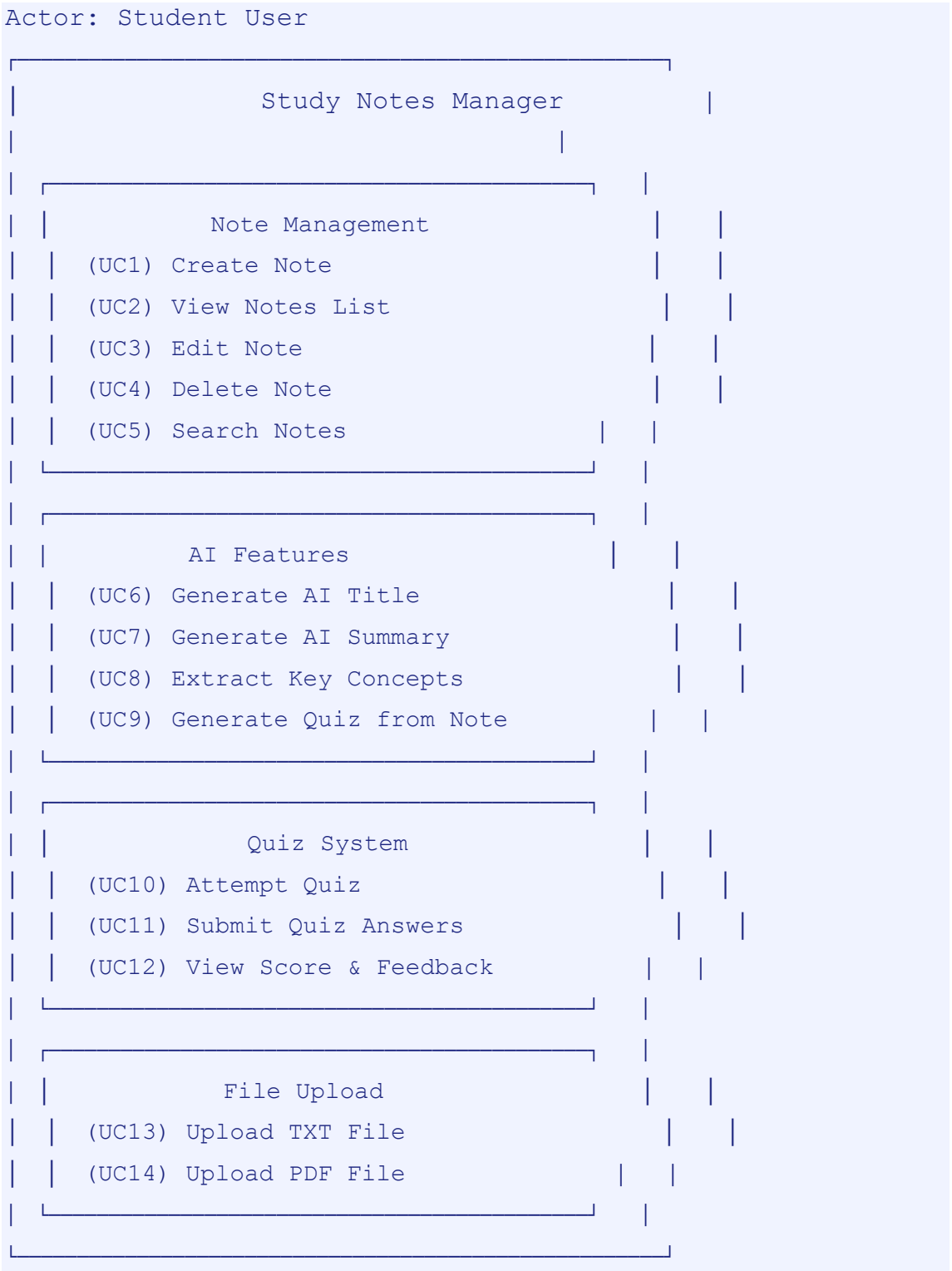
2. System Design

2.1 System Architecture Diagram

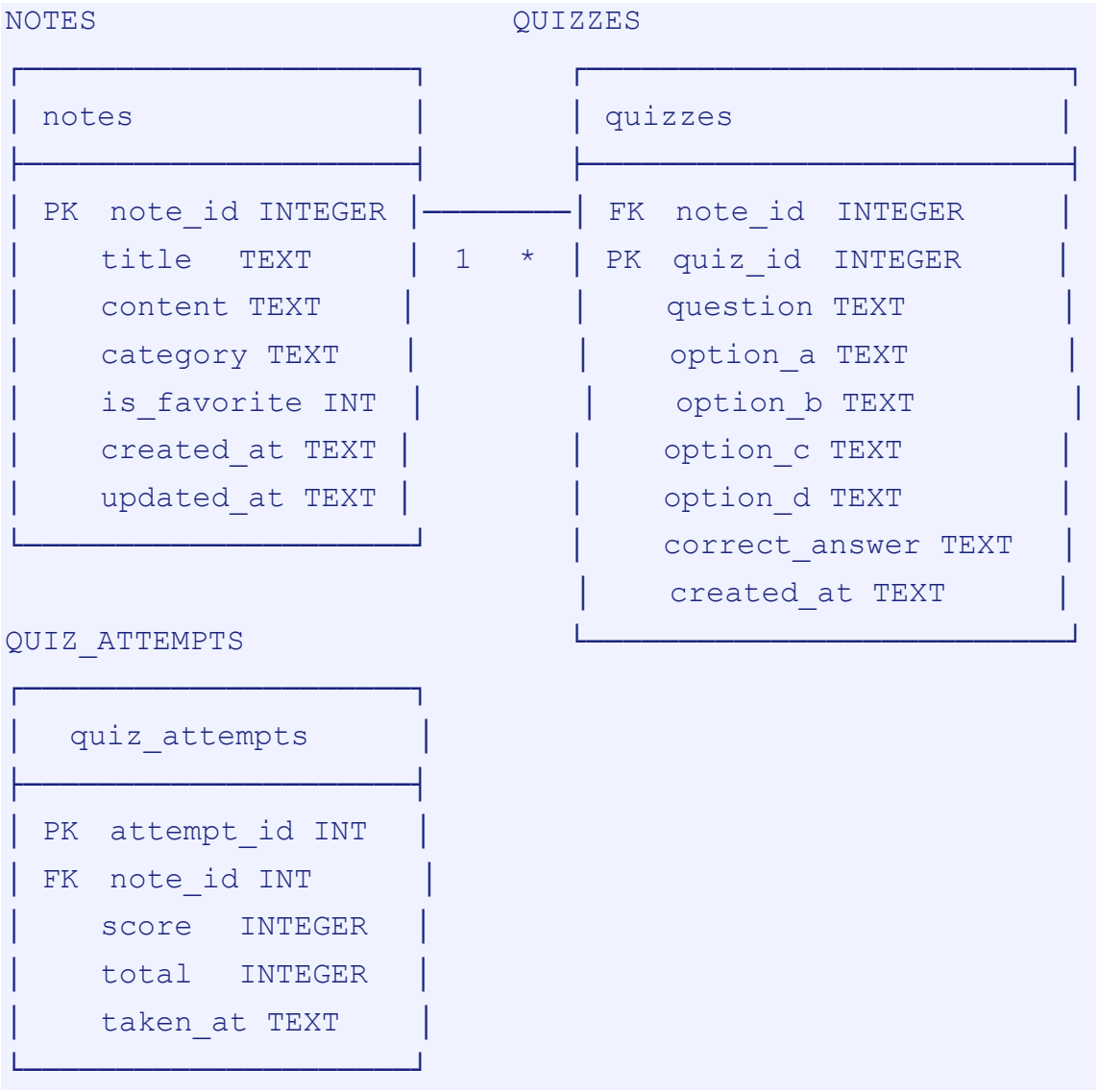
The application follows the MVC (Model-View-Controller) pattern with a clear separation between the presentation layer, business logic, and data persistence.



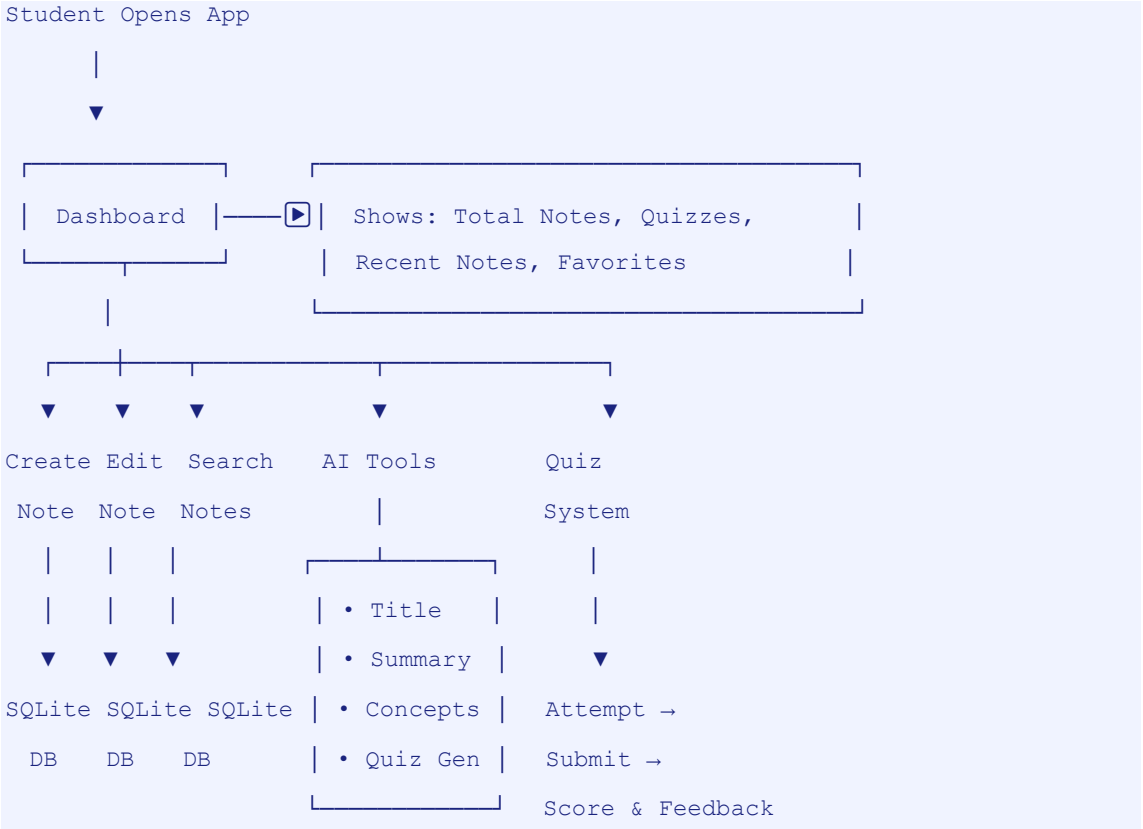
2.2 Use Case Diagram



2.3 Database ER Diagram



2.4 Application Workflow



3.Implementation

This starting code sets up the basic environment for your AI-powered Study Notes Manager project. It imports all the necessary libraries needed for the application. Flask is used to build the web application and handle requests from the frontend. SQLite is used as the database to store notes, summaries, quizzes, or user data. The `dotenv` package loads secret information like the OpenAI API key from a `.env` file for security.

The project also imports the OpenAI library so AI features like note summarization, title generation, and quiz creation can be added. `PyPDF2` helps read text from PDF files, while `PIL` and `pytesseract` are used for OCR (Optical Character Recognition), which extracts

```
import os
import sqlite3
import json
import io
from dotenv import load_dotenv
load_dotenv()
from datetime import datetime
from flask import Flask, request, jsonify, render_template, g
from openai import OpenAI
import PyPDF2
from PIL import Image
import pytesseract

import platform
if platform.system() == "Windows":
    pytesseract.pytesseract.tesseract_cmd = r'C:\Program
Files\Tesseract-OCR\tesseract.exe'
else:
    pytesseract.pytesseract.tesseract_cmd = '/usr/bin/tesseract'
```

text from uploaded images. Finally, the platform module checks whether the system is Windows or Linux and sets the correct Tesseract OCR path accordingly, ensuring OCR works properly on different operating systems.

The OpenAI() part initializes the AI client using the DeepSeek API key stored in the environment variables. The base_url="https://api.deepseek.com" tells the program to use DeepSeek's API instead of the normal OpenAI API server. This client will later be used for AI features like generating summaries, quizzes, titles, or answers.

```
# DeepSeek client setup
client = OpenAI(
    api_key=os.environ.get('DEEPSEEK_API_KEY'),
    base_url="https://api.deepseek.com"
)

app = Flask(__name__)
```

api_key=os.environ.get('DEEPSEEK_API_KEY')

This line safely reads the API key from the `.env` file or system environment.

app = Flask(__name__)

This creates the main Flask web application object. It is the core of the backend server and is used to define routes, handle requests, and run the website.

```
@app.after_request
def add_headers(response):
    response.headers['Content-Security-Policy'] = "default-src 'self';
script-src 'self' 'unsafe-eval' 'unsafe-inline'; style-src 'self' 'unsafe-
inline'"
    return response

app.config['MAX_CONTENT_LENGTH'] = 20 * 1024 * 1024
DB_PATH = 'notes.db'

# ── Database ──────────────────────────────────────────────

def get_db():
    if 'db' not in g:
        g.db = sqlite3.connect(DB_PATH)
        g.db.row_factory = sqlite3.Row
    return g.db

@app.teardown_appcontext
def close_db(e=None):
    db = g.pop('db', None)
    if db:
        db.close()
```

This part of the code handles security settings, file upload limits, and database connection management for the Flask application.

The @app.after_request function runs after every response is sent from the server. It adds a Content-Security-Policy header to improve website security by controlling which scripts and styles are allowed to run in the browser. This helps reduce risks like malicious script injection attacks.

app.config ['MAX_CONTENT_LENGTH'] = 20 * 1024 * 1024

This sets the maximum upload file size to 20 MB, preventing users from uploading very large files.

DB_PATH = 'notes.db'

This stores the SQLite database file name.

The get_db() function creates a database connection only when needed and stores it inside Flask's g object so the same connection can be reused during a request.
sqlite3.Row allows database rows to behave like dictionaries, making column access easier.

Finally, the close_db() function automatically closes the database connection after each request is finished. This helps manage resources properly and prevents connection leaks.

3.1 Core Code Snippets

AI Integration with DeepSeek API:

```
def ai_extract_text(file_bytes, media_type, filename):
    """Extract text from PDF or image files"""
    try:
        if media_type == 'application/pdf':
            # Extract text from PDF
            pdf_reader = PyPDF2.PdfReader(io.BytesIO(file_bytes))
            text = ""
            for page in pdf_reader.pages:
                extracted = page.extract_text()
                if extracted:
                    text += extracted + "\n"
            if not text.strip():
                raise Exception("No text could be extracted from PDF")
            return text.strip()
```

```
        elif media_type.startswith('image/'):
            # Extract text from image using Tesseract OCR
            img = Image.open(io.BytesIO(file_bytes))
            # Preprocess image for better OCR results
            img = img.convert('L')  # Convert to grayscale
            text = pytesseract.image_to_string(img)
            if not text.strip():
                raise Exception("No text could be extracted from image")
            return text.strip()

        else:
            raise Exception(f"Unsupported file type: {media_type}")

    except Exception as e:
        raise Exception(f"Text extraction failed: {str(e)}")
```

This function handles PDF and image text extraction. PyPDF2 extracts text from PDFs, while Tesseract OCR processes images after converting them to grayscale for better accuracy.

3.2 Major Challenges & Solutions

Challenge 1: Extracted Text Had Unwanted Spacing

Problem: PDF extraction resulted in random spaces and line breaks.

Solution: Created a `clean_extracted_text()` function using regex to normalize spacing and line breaks.

Challenge 2: Missing Quiz Explanations

Problem: AI-generated quiz had no explanations for answers.

Solution: Updated the API prompt to require a mandatory explanation field in

the JSON response.

```
@app.route('/api/quiz/check', methods=['POST'])
def check_quiz_answer():
    """Check if selected answer is correct"""
    try:
        data = request.json
        note_id = data.get('note_id')
        question_index = data.get('question_index')
        selected_answer = data.get('answer')

        if note_id is None or question_index is None or selected_answer is
None:
            return jsonify({'error': 'Missing required fields'}), 400

        db = get_db()
        row = db.execute("SELECT ai_quiz FROM notes WHERE id=?",
(note_id,)).fetchone()

        if not row:
            return jsonify({'error': 'Note not found'}), 404

        if not row['ai_quiz']:
            return jsonify({'error': 'No quiz found for this note'}), 404

        quiz = json.loads(row['ai_quiz'])

        if question_index >= len(quiz):
            return jsonify({'error': 'Invalid question index'}), 400

        correct_answer = quiz[question_index]['answer']
        is_correct = (selected_answer == correct_answer)

        return jsonify({
            'correct': is_correct,
            'correct_answer_index': correct_answer,
            'correct_answer_text':
quiz[question_index]['options'][correct_answer]
        })
```

```
    except Exception as e:
        print(f"Quiz check error: {e}")
        return jsonify({'error': str(e)}), 500
```

Challenge 3: Timer Didn't Auto-Submit

Problem: Quiz timer expired but answers remained active.

Solution: Added `endQuiz()` function that disables all radio buttons and calculates score automatically.

```
@app.route('/api/quiz/<int:note_id>/explanation/<int:question_index>',
methods=['GET'])
def get_quiz_explanation(note_id, question_index):
    """Get explanation for a specific quiz question"""
    try:
        db = get_db()
        row = db.execute("SELECT ai_quiz FROM notes WHERE id=?",
(note_id,)).fetchone()

        if not row or not row['ai_quiz']:
            return jsonify({'error': 'No quiz found'}), 404

        quiz = json.loads(row['ai_quiz'])

        if question_index >= len(quiz):
            return jsonify({'error': 'Invalid question index'}), 400

        question_data = quiz[question_index]

        return jsonify({
            'question': question_data['question'],
            'correct_answer':
question_data['options'][question_data['answer']],
            'explanation': question_data.get('explanation', 'No explanation
available for this question.')
        })
```

```
    except Exception as e:
        print(f"Explanation error: {e}")
        return jsonify({'error': str(e)}), 500
```

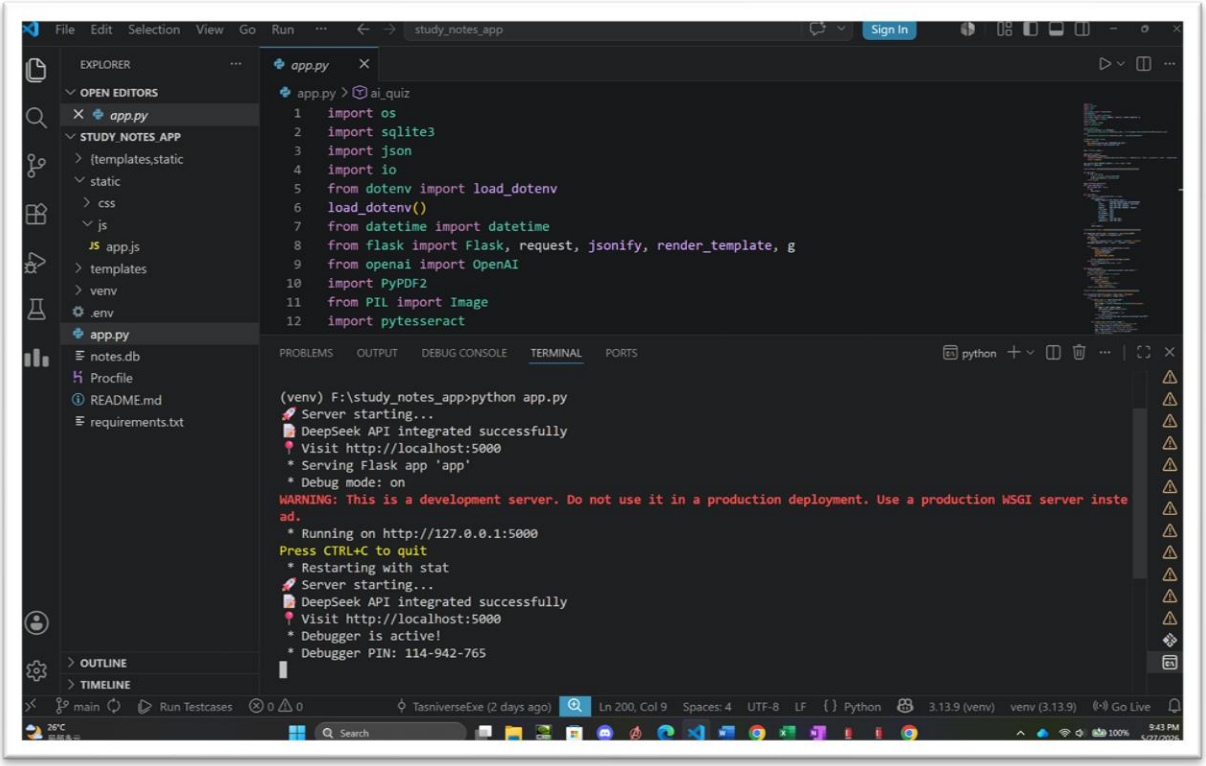
4. Testing and Results

The system was tested using different types of study materials, including typed notes, PDF files, and images containing text. Several features such as note uploading, OCR text extraction, AI-generated summaries, title generation, and quiz creation were tested individually to ensure proper functionality.

During testing, the OCR system successfully extracted readable text from most uploaded images, while PDF processing worked correctly for text-based PDF documents. The AI features generated meaningful summaries, relevant titles, and quiz questions based on the uploaded study content. Database operations such as storing and retrieving notes also functioned properly.

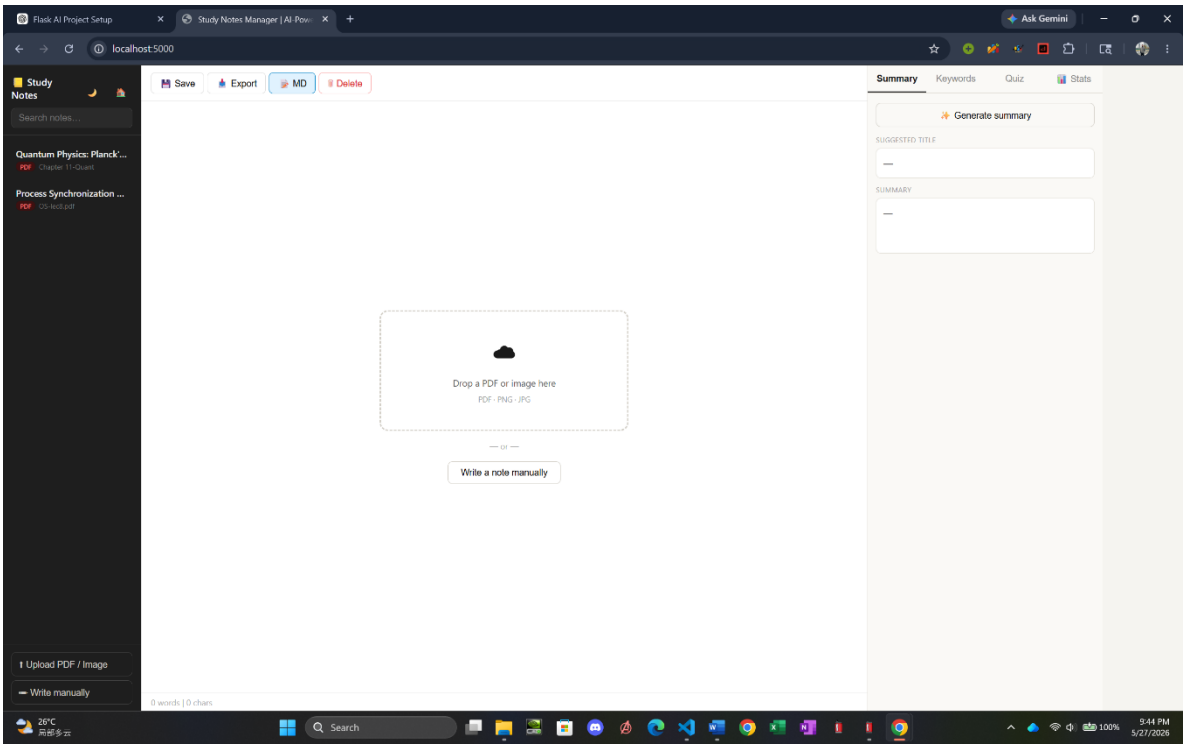
The application showed stable performance for normal-sized files and responded quickly to most user requests. Some minor issues, such as inaccurate OCR results for unclear images and inconsistent quiz explanations, were identified and improved during development.

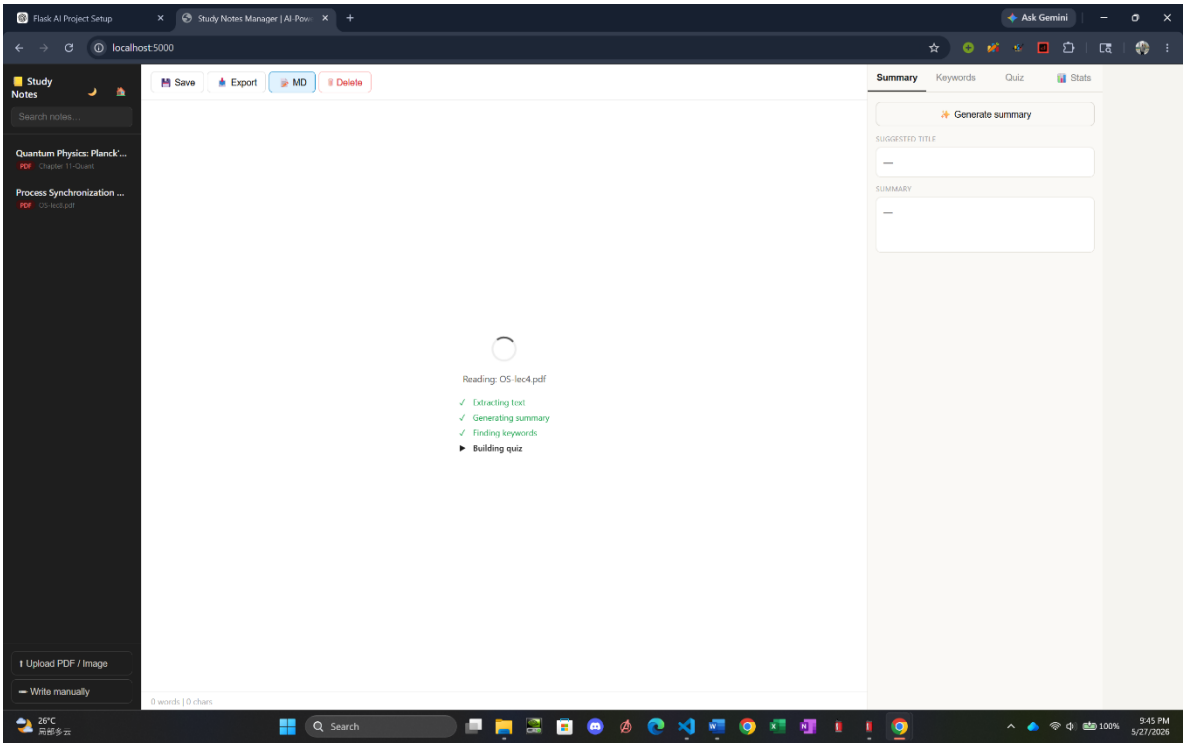
Overall, the testing results demonstrated that the system can effectively manage study notes and provide AI-assisted learning features in an organized and user-friendly way.



When a user uploads a PDF file, the system uses the PyPDF2 library to read each page and extract text line by line. For image files such as PNG or JPG, the system uses the Tesseract OCR engine. Before extracting text from an image, the image is first converted to grayscale. This preprocessing step improves the accuracy of the OCR. After extraction, the raw text is cleaned by removing extra spaces and unnecessary line breaks. This cleaned text is then sent to the AI module for further processing.

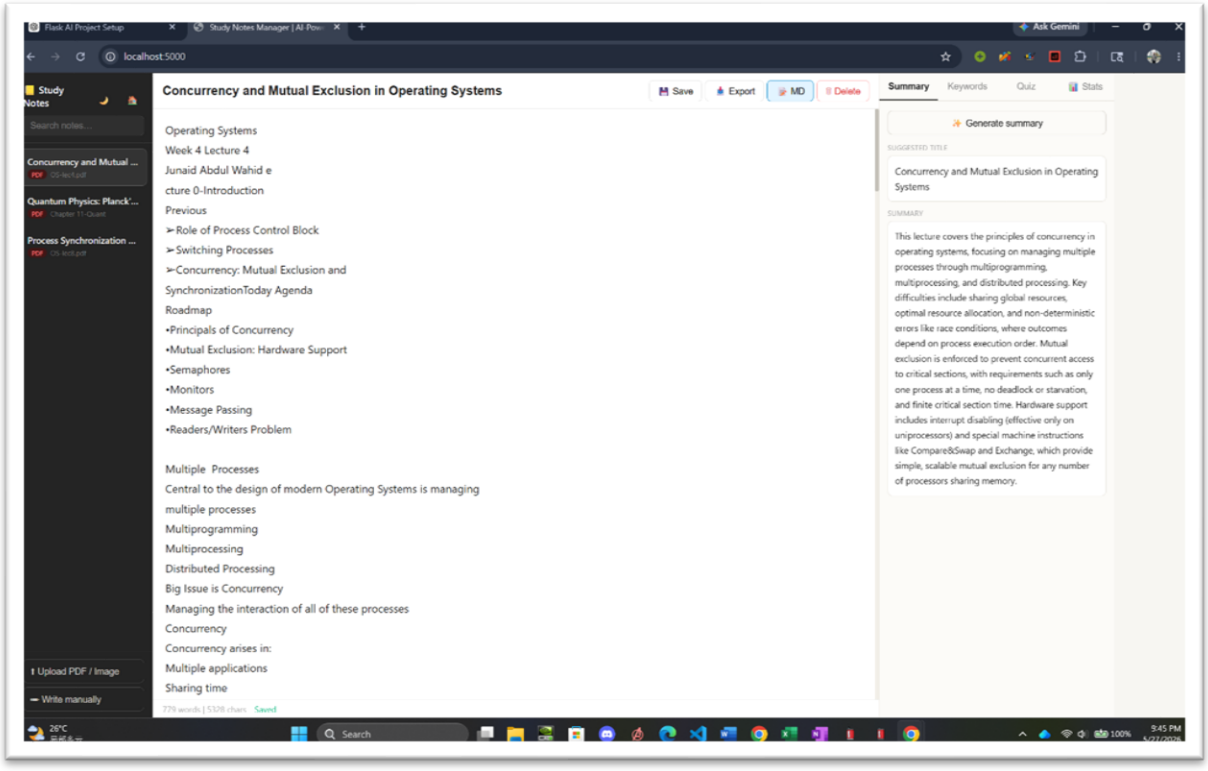
PDF and Image Text Extraction

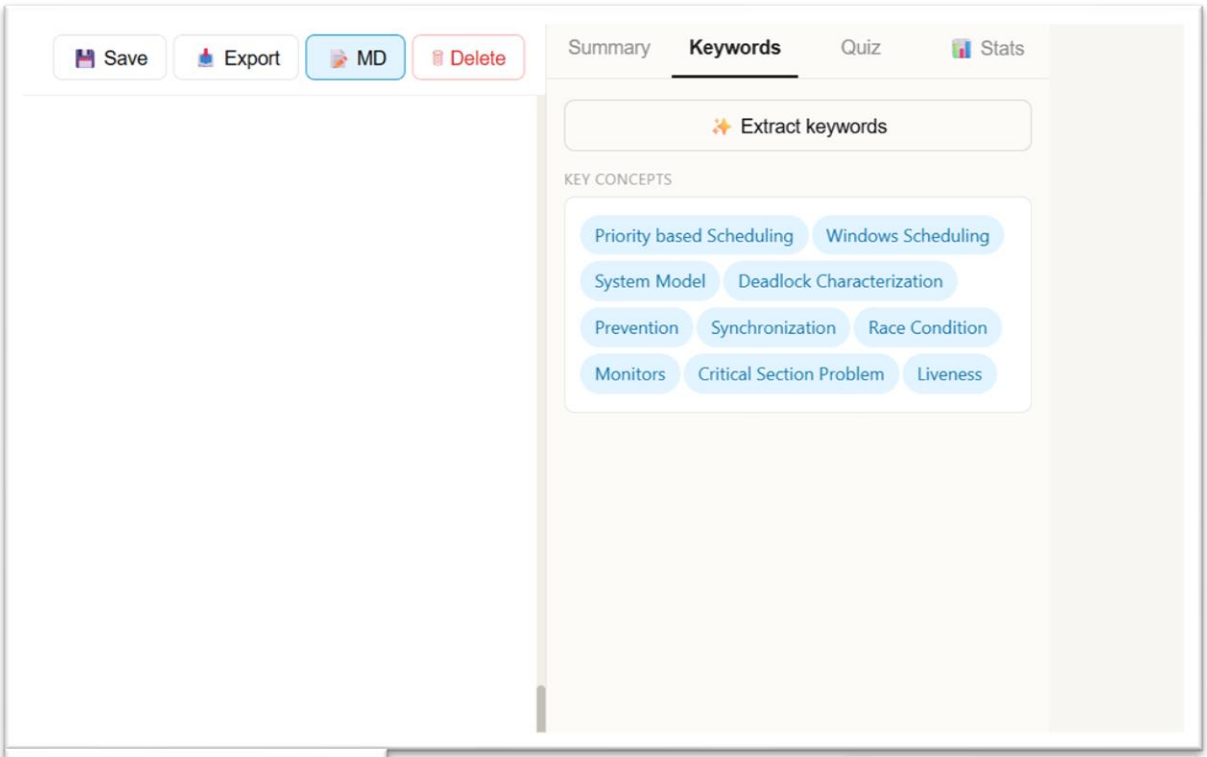


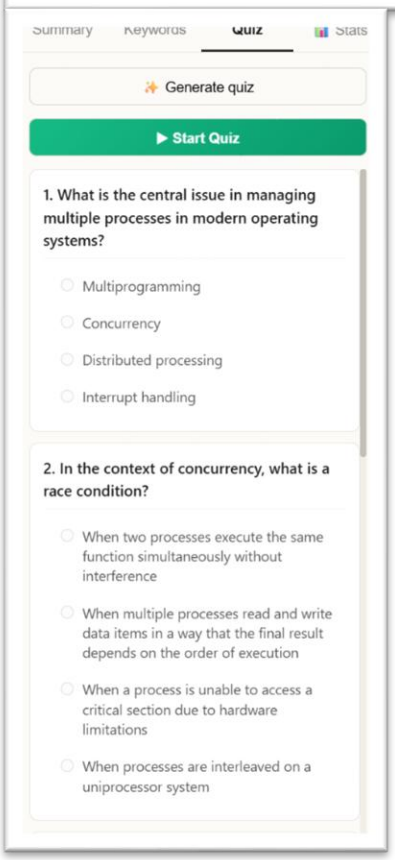


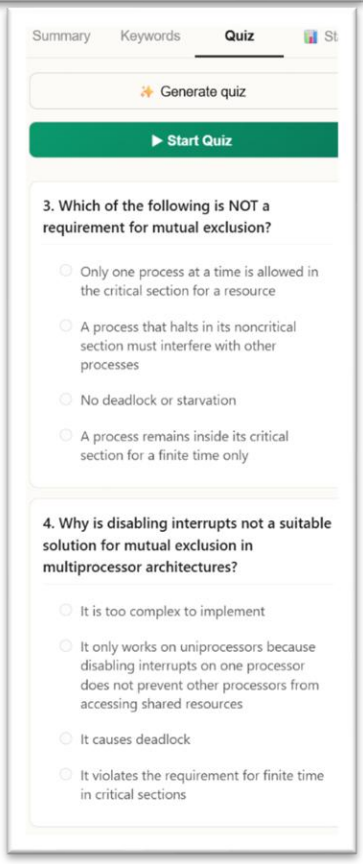
AI-Powered Content Generation Using DeepSeek API

The system integrates the DeepSeek API to generate summaries, keywords, and quizzes from the extracted text. A system prompt is designed to instruct the AI to return only valid JSON output. The temperature parameter is set to 0.7, which balances creativity and accuracy. Three separate API calls are made: one for generating a title and summary, one for extracting key concepts, and one for creating a 10-question multiple-choice quiz with explanations. Each AI response is parsed and stored in the database along with the original note.




Save
Export
MD
Delete
Summary
Keywords
Quiz
Stats
Extract keywords
KEY CONCEPTS
Priority based Scheduling
Windows Scheduling
System Model
Deadlock Characterization
Prevention
Synchronization
Race Condition
Monitors
Critical Section Problem
Liveness

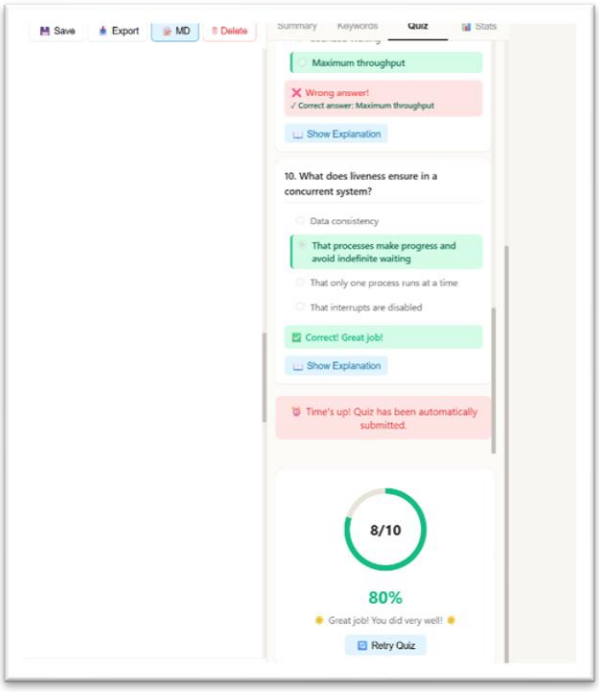

Summary
Keywords
Quiz
Stats
Generate quiz
▶ Start Quiz
1. What is the central issue in managing multiple processes in modern operating systems?
Multiprogramming
Concurrency
Distributed processing
Interrupt handling
2. In the context of concurrency, what is a race condition?
When two processes execute the same function simultaneously without interference
When multiple processes read and write data items in a way that the final result depends on the order of execution
When a process is unable to access a critical section due to hardware limitations
When processes are interleaved on a uniprocessor system


Summary
Keywords
Quiz
Generate quiz
▶ Start Quiz
3. Which of the following is NOT a requirement for mutual exclusion?
Only one process at a time is allowed in the critical section for a resource
A process that halts in its noncritical section must interfere with other processes
No deadlock or starvation
A process remains inside its critical section for a finite time only
4. Why is disabling interrupts not a suitable solution for mutual exclusion in multiprocessor architectures?
It is too complex to implement
It only works on uniprocessors because disabling interrupts on one processor does not prevent other processors from accessing shared resources
It causes deadlock
It violates the requirement for finite time in critical sections

Quiz Timer and Score Circle System

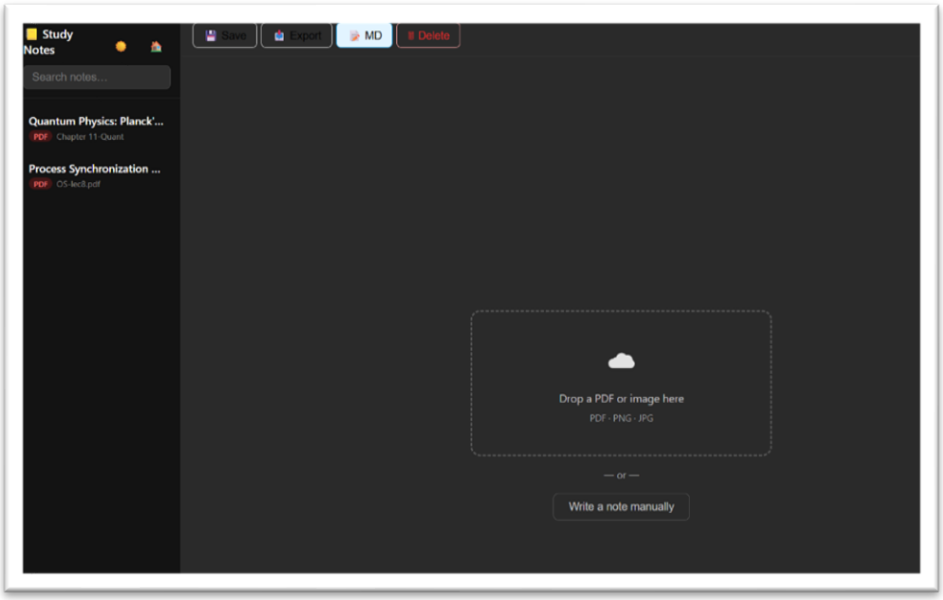
When the user clicks the "Start Quiz" button, a JavaScript timer begins counting down. The timer duration is calculated based on the number of questions (6 seconds per question). Every second, the timer display updates. A progress bar visually shows the remaining time. When the timer reaches zero or when the user answers all questions, the quiz automatically ends. At that point, all radio buttons are disabled, and the score is calculated.

The score is displayed using an SVG circle. The circle fills based on the percentage of correct answers. The color of the circle changes according to the score: green for 80% or above, orange for 60% to 79%, yellow for 40% to 59%, and red for below 40%. A message is also displayed to encourage the user based on their performance.



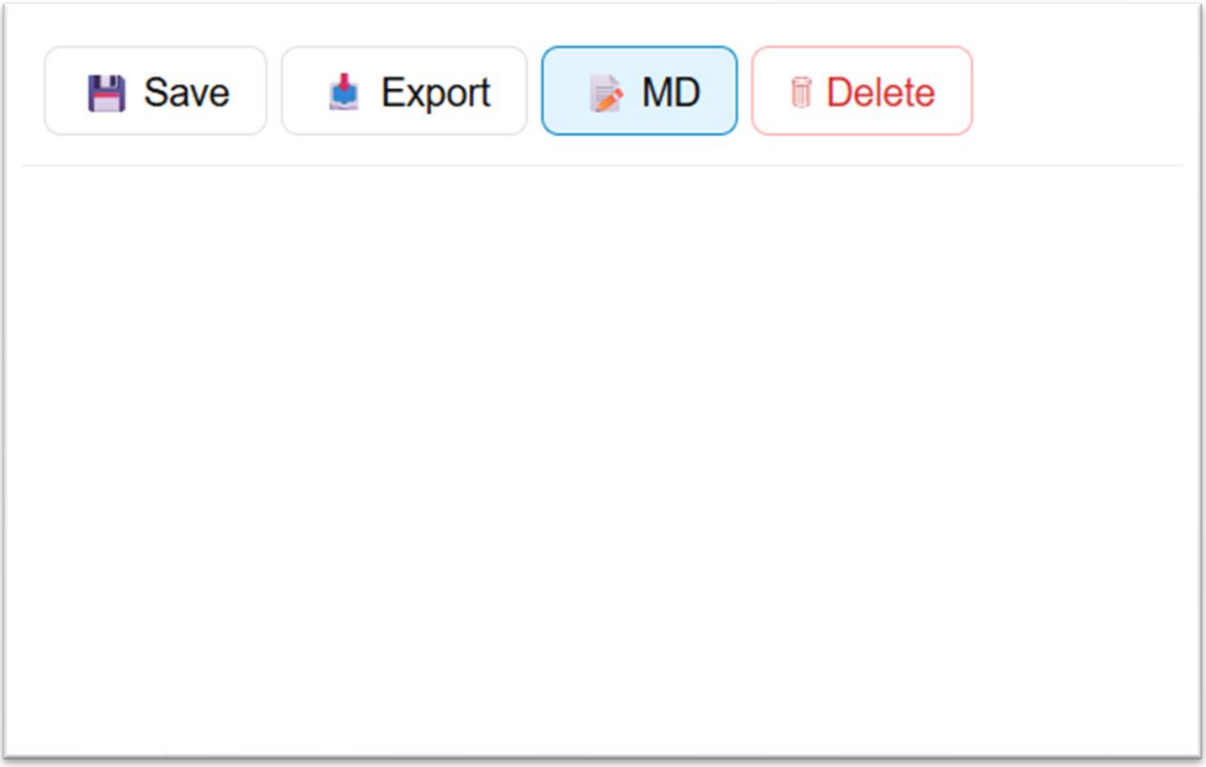
Dark Mode with Local Storage

The system includes a dark mode feature for better user experience during nighttime study sessions. When the user clicks the theme toggle button (moon/sun icon), the dark-mode class is added or removed from the body element. The user's preference is saved in the browser's local storage. When the user visits the website again, the system checks the saved preference and automatically applies the correct theme. This ensures a consistent experience across sessions.



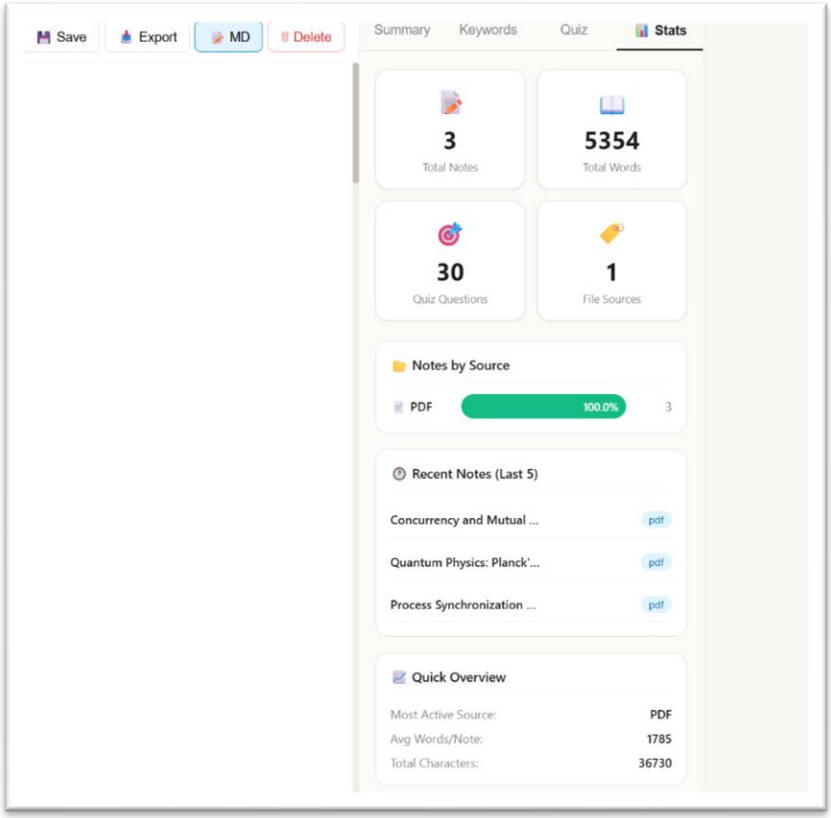
Markdown Live Preview

The note editor supports Markdown formatting. Users can write notes using Markdown syntax such as headings, bold, italic, lists, and code blocks. When the user clicks the "MD" button, the editor is hidden and a preview panel is shown. The Marked.js library converts the Markdown text to HTML in real time. The preview updates automatically as the user types. This feature allows users to see how their formatted note will look before saving or exporting it.



Analytics Dashboard

The analytics dashboard provides users with insights into their study habits. When the dashboard tab is opened, the system calculates the total number of notes, total word count across all notes, total characters, and average words per note. It also shows a source distribution chart displaying how many notes came from manual entry, PDF uploads, and image uploads. A list of the five most recently updated notes is displayed, and users can click any recent note to open it directly. All statistics are updated automatically whenever a note is added, edited, or deleted.



Conclusion

The AI-Powered Study Notes Manager is a complete, functional, and user-friendly web application that successfully achieves its primary objectives. It allows students, teachers, professors, and business professionals to upload PDF or image files, extract text automatically, and generate AI-powered summaries, keywords, and quizzes.

The project demonstrates the effective integration of multiple technologies. The Flask framework provides a robust backend with RESTful API endpoints. The SQLite database stores notes along with AI-generated content such as titles, summaries, keywords, and quizzes. The DeepSeek API enables intelligent content generation, including detailed summaries, key concept extraction, and multiple-choice questions with explanations.

From a frontend perspective, the application offers a clean and intuitive three-column layout. Users can write notes using Markdown with live preview, switch between light and dark mode, export notes as TXT or PDF, and delete notes when no longer needed. The quiz system includes a timer, a progress bar, a start button, and a visual score circle that provides immediate feedback. The analytics dashboard gives users insights into their total notes, word count, source distribution, and recent activity.

Several challenges were encountered during development, including inaccurate text extraction from PDFs, missing explanations in AI-generated quizzes, and timer synchronization issues. These were solved by implementing a text cleaning function, updating the AI prompt to require explanations, and adding proper JavaScript timer management with auto-submit functionality.

The application is deployed on Render.com and is accessible online. The source code is publicly available on GitHub for future improvements and collaboration.

Looking ahead, there are many opportunities to enhance this project. Future developments may include voice input for hands-free note taking, real-time collaborative editing, a dedicated mobile application, multi-language support, advanced analytics with progress tracking, reminders, tags and folders, note sharing via links, undo and redo functionality, and interactive tutorials for new users.

In summary, this project successfully combines artificial intelligence with modern web development to create a practical and valuable tool for learning and knowledge management. It fulfills all the requirements of a final year project and demonstrates the potential of AI-powered educational tools.

References:

AI & APIs

DeepSeek API. (2025). DeepSeek Platform.

Available at: https://platform.deepseek.com/

OpenAI Python Library. (2025). OpenAI API Client for Python.

Available at: https://github.com/openai/openai-python

Python Libraries

Flask Documentation. (2025). Flask Web Framework.

Available at: https://flask.palletsprojects.com/

SQLite3 Documentation. (2025). SQLite3 – DB-API 2.0 Interface for SQLite.

Available at: https://docs.python.org/3/library/sqlite3.html

PyPDF2 Documentation. (2025). PyPDF2 – PDF Processing in Python.

Available at: https://pypi.org/project/PyPDF2/

Pillow (PIL) Documentation. (2025). Python Imaging Library.

Available at: https://python-pillow.org/

Pytesseract Documentation. (2025). Python Tesseract Wrapper.

Available at: https://pypi.org/project/pytesseract/

Tesseract OCR Engine. (2025). Open Source OCR Engine.

Available at: https://github.com/tesseract-ocr/tesseract

python-dotenv Documentation. (2025). Environment Variable Management.

Available at: https://pypi.org/project/python-dotenv/

Gunicorn Documentation. (2025). Python WSGI HTTP Server.

Available at: https://gunicorn.org/

Frontend Libraries

Marked.js Documentation. (2025). Markdown Parser and Compiler.

Available at: https://marked.js.org/

CodeMirror Documentation. (2025). Browser-based Text Editor.

Available at: https://codemirror.net/

Hosting and Version Control

Render Documentation. (2025). Cloud Hosting Platform.

Available at: https://render.com/

Git Documentation. (2025). Distributed Version Control System.

Available at: https://git-scm.com/

GitHub. (2025). Code Repository Hosting Service.

Available at: https://github.com/

GitHub Desktop. (2025). GUI for Git Management.

Available at: https://desktop.github.com/

Development Tools

Visual Studio Code. (2025). Code Editor.

Available at: https://code.visualstudio.com/

Python Software Foundation. (2025). Python Programming Language (Version 3.10+). Available at: https://www.python.org/

Postman. (2025). API Testing Tool. Available at: https://www.postman.com/

Chrome DevTools. (2025). Browser-based Debugging Tools.

Available at: https://developer.chrome.com/docs/devtools/

GitHub Repository

MD TASNIMUL HASAN. (2025). Study Notes Manager with AI Assistance – Source Code.

Available at: https://github.com/TasnimShaun/Study-Notes-Manager-with-AI-Assistance

Project Deploy: https://study-notes-manager.onrender.com

Self Contribution (Original Work)

The following components were designed and implemented entirely by the developer:

- Complete three-layer system architecture (Frontend, Backend, Database)
- AI prompt engineering for DeepSeek API (summary, keywords, quiz, explanation generation)
- Quiz timer system with auto-submit functionality and visual score circle using SVG
- Dark mode with localStorage persistence across browser sessions
- Markdown live preview integration using Marked.js library
- Analytics dashboard for study statistics (total notes, word count, source distribution, recent notes)
- Text cleaning algorithm using regular expressions for extracted PDF content
- Responsive CSS layout supporting both desktop and mobile devices using media queries
- Home button with full application state reset
- Complete user interface and user experience design (three-column layout: Sidebar, Editor, AI Panel)
- Error handling for API calls, file upload operations, and database transactions
- Flask application deployment on Render.com with Tesseract OCR support